

# Jobs Page & Card Component

---

Let's start off with the `/jobs` page. Right now, it just shows a list of titles. We want to have a nice detailed card for each job listing. We will also add a search form at the top of the page but that functionality will be added later.

We will create a new Blade component for the job card. This will allow us to reuse the card in multiple places. We will also update the jobs page to use this new component.

Open the `resources/views/components/job-card.blade.php` file. The HTML for this can be found in the `_theme_files/jobs.html`.

We need the grid wrapping markup and the card markup. Let's add the grid wrapper first. In the `resources/views/jobs/index.php` file, add the following code:

```
<x-layout>
  <x-slot:pageTitle>All Jobs</x-slot:pageTitle>
  <h1 class="text-2xl">{{ $title }}</h1>
  <div class="grid grid-cols-1 md:grid-cols-3 gap-4 mb-6">
    @forelse($jobs as $job)
      <div>{{ $job->title }}</div>
    @empty
      <p>No jobs found</p>
    @endforelse
  </div>
</x-layout>
```

Instead of an unordered list, we will use a grid with 3 columns. This will allow us to have 3 jobs per row. Right now it is only showing the title. We want to replace that `<div>{{ $job->title }}</div>` with the job card component.

## Job Card Component

Let's create a new Blade component for the job card. Run the following command to create a new Blade component:

```
php artisan make:component JobCard
```

Open the `resources/views/components/job-card.blade.php` file. and add the following code for now:

```
@props(['job'])
<div class="rounded-lg shadow-md bg-white p-4">{{ $job->title }}</div>
```

This is a very basic card that takes in the job as a prop. We will add more details to it later. For now, let's update the `resources/views/jobs/index.php` file to use this new component.

Replace the `<div>{{ $job->title }}</div>` with the following code:

```
<x-job-card :job="$job" />
```

Now you should see the titles with the background color and rounded corners.

Let's add the rest of the job details and markup:

```
@props(['job'])
<div class="rounded-lg shadow-md bg-white p-4">
  <div class="flex items-center space-between gap-4">
    company_name }}"
      class="w-14"
    />
    <div>
      <h2 class="text-xl font-semibold">{{ $job->title }}</h2>
      <p class="text-sm text-gray-500">{{ $job->job_type }}</p>
    </div>
  </div>
  <p class="text-gray-700 text-lg mt-2">
    {{ Str::limit($job->description, 100) }}
  </p>
  <ul class="my-4 bg-gray-100 p-4 rounded">
    <li class="mb-2">
      <strong>Salary:</strong> {{ number_format($job->salary) }}
    </li>
    <li class="mb-2">
      <strong>Location:</strong> {{ $job->city }}, {{ $job->state }} @if
        ($job->remote)
      <span class="text-xs bg-green-500 text-white rounded-full px-2 py-1 ml-2">
        Remote
      </span>
    @else
      <span class="text-xs bg-red-500 text-white rounded-full px-2 py-1 ml-2">
        On-site
      </span>
    @endif
    </li>
    <li class="mb-2">
      <strong>Tags:</strong> {{ucwords(str_replace(', ', ' ', $job->tags))}}
    </li>
  </ul>
  <a
    href="{{ route('jobs.show', $job->id) }}"
    class="block w-full text-center px-5 py-2.5 shadow-sm rounded border text-base
```

```
font-medium text-indigo-700 bg-indigo-100 hover:bg-indigo-200"
>
  Details
</a>
</div>
```

We have just changed the hardcoded values to use the job properties. We have also added a link to the job details page. For the description, I used the `Str::limit()` function to limit the description to 100 characters. Other than that, everything is pretty straightforward.

## Logo

The company logo is not showing because they are not in the project. Open the `_theme_files/images` folder and copy the entire `logos` folder to the `public/images/logos` folder. This will allow the images to be accessible from the browser and you should now see the logos.

We will handle the search form later on as well as pagination.

In the next lesson, we will handle displaying the job listings on the homepage.